

Postgres Deep Dive: The Working Reference

Everything I keep needing to look up about Postgres — schema design, indexes, JSON/JSONB, full-text search, EXPLAIN interpretation, transaction isolation, common anti-patterns — in one printable reference. Targeted at the solo developer who's past "I know SQL" and into "I want to use Postgres properly."

NOTE. LAST VALIDATED: 2026-05. Postgres 16 (current LTS-equivalent), `psql` 16, pgvector 0.7+, TimescaleDB 2.x, PostGIS 3.4+. Bump and re-validate per [CLAUDE.md](#) standard #5.

What this doc is for

A working reference, not a tutorial. Read sections as needed; the cheat sheet at the end is what you print and keep at your desk. Everything assumes you've used Postgres before — connected, ran queries, made a table. The gap this fills is "I've used Postgres but never quite owned it."

Prerequisites

- A running Postgres 14+ (16+ recommended). The shared Postgres from [vps-from-zero](#) works.
 - `psql` installed locally (`apt install postgresql-client` or via your distro).
 - Comfort with basic SQL (SELECT, INSERT, UPDATE, DELETE, JOIN).
-

Table of contents

1. [Mental model and vocabulary](#)
2. [psql mastery](#)
3. [Schema design fundamentals](#)
4. [Data types: the full menu](#)
5. [Indexes: every type, when to use](#)
6. [Query patterns worth knowing](#)
7. [JSON / JSONB deep dive](#)
8. [Full-text search](#)
9. [Constraints beyond NOT NULL](#)
10. [Transactions and isolation](#)
11. [Locks: explicit and implicit](#)
12. [Triggers and PL/pgSQL](#)
13. [Views and materialized views](#)
14. [EXPLAIN: reading query plans](#)
15. [Performance: statistics, autovacuum, slow queries](#)
16. [Extensions worth knowing](#)
17. [postgresql.conf knobs that matter](#)
18. [Connection pooling: pgbouncer](#)

- 19. [Replication concepts](#)
 - 20. [Anti-patterns to know and avoid](#)
 - 21. [Cheat sheet](#)
-

1. Mental model and vocabulary

MVCC: how Postgres handles concurrent reads and writes

Postgres uses **MVCC** (Multi-Version Concurrency Control). When you `UPDATE` a row, Postgres doesn't overwrite it in place — it writes a new version of the row and marks the old one as dead. Readers continue to see the old version until their transaction ends; writers see (and produce) new versions.

Consequences:

- **Readers never block writers, and writers never block readers.** This is the headline MVCC property. You can run a long `SELECT` while heavy writes are happening; neither stops the other.
- **Dead rows accumulate.** They're cleaned up by **autovacuum** (background process). If autovacuum can't keep up, tables and indexes bloat.
- **Sequential IDs (`SERIAL` / `IDENTITY`) can have gaps.** A failed `INSERT` still consumes a sequence value. This is fine; don't treat IDs as audit-perfect counters.
- **`TXID` (transaction ID) is a 32-bit counter that wraps.** Postgres has to "freeze" old transaction IDs periodically. If freezing falls behind, you get the "wraparound" disaster scenario. Mostly handled automatically by autovacuum, but worth knowing the term.

WAL: the durability story

Every change is first written to the **Write-Ahead Log (WAL)** before being applied to data files. If the server crashes mid-write, WAL replay on restart brings the database to a consistent state. WAL also drives streaming replication.

Implications:

- A `COMMIT` returns after the WAL record is flushed to disk. This is the durability bottleneck for write-heavy workloads.
- If you ever set `synchronous_commit = off`, `COMMIT` becomes fast but you can lose recent commits on a crash. Don't do this for personal/production unless you genuinely understand the tradeoff.
- WAL files accumulate until they're archived (for PITR) or recycled. Filesystem fills up with WAL = service stops accepting writes. Watch `pg_stat_wal`.

Pages and tuples

Storage unit on disk is an **8KB page**. Rows are **tuples** within pages. `TOAST` handles oversized values (large strings, `JSONB`) by splitting them across out-of-band storage. Mostly transparent; matters for performance (sequential scans walk pages).

Catalogs

Postgres's own metadata lives in **system catalogs** (`pg_class`, `pg_attribute`, `pg_index`, etc.) and is queryable with regular SQL. The same SELECT you'd use on your own tables works on `pg_class`. This is incredibly useful for "find me all tables larger than 1GB" or "which indexes haven't been used in a month."

```
-- Tables sorted by size
SELECT relname,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
       pg_size_pretty(pg_relation_size(relid)) AS table_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Versioning

- **Major versions** (16 → 17): data file format may change; requires `pg_upgrade` or dump/restore. Released annually each fall.
 - **Minor versions** (16.3 → 16.4): bugfixes, no format changes. Released quarterly. Always-apply.
- Pin major version in your compose file (`postgres:16`). Never use `:latest`.

2. `psql` mastery

`psql` is the canonical CLI client. Learn it. Half the productive-Postgres-user advantage is fluency here.

Connecting

```
psql                                # connect to default DB as current user
psql -d mydb                        # specific DB
psql -h hostname -p 5432 -U user -d mydb # full connection
psql 'postgresql://user:pass@host:5432/db' # connection URI
```

Inside a Docker setup like the one from [vps-from-zero](#):

```
docker compose exec postgres psql -U postgres
docker compose exec postgres psql -U appuser -d appdb
```

Backslash commands (meta-commands)

Command	Shows
<code>\l</code>	List databases
<code>\l+</code>	List databases with size, owner, description
<code>\c dbname</code>	Connect to a different database
<code>\dt</code>	List tables in current schema
<code>\dt *.*</code>	List tables in all schemas
<code>\dt+</code>	Tables with size info
<code>\d tablename</code>	Describe a table (columns, types, constraints, indexes)
<code>\d+ tablename</code>	Describe with extra detail (storage, comments)
<code>\du</code>	List users / roles
<code>\du+ rolename</code>	Detailed role info including memberships
<code>\dn</code>	List schemas
<code>\df</code>	List functions
<code>\dv</code>	List views
<code>\di</code>	List indexes
<code>\dx</code>	List installed extensions
<code>\ds</code>	List sequences
<code>\dp</code> or <code>\z</code>	Show table privileges
<code>\timing on</code>	Show query execution time after each query
<code>\x</code>	Toggle "expanded" output (one row per page, fields stacked) — invaluable for wide tables
<code>\e</code>	Edit the last query in <code>\$EDITOR</code>
<code>\i file.sql</code>	Execute a file
<code>\copy table FROM 'file.csv' WITH CSV HEADER</code>	Bulk import (client-side equivalent of COPY)
<code>\q</code>	Quit

Output formatting

```
\x                -- expanded output toggle
\x on             -- always expanded
\x auto           -- expanded only when needed (the best default)
\pset format wrapped -- wrap long values across multiple lines
\pset null 'Ø'    -- show NULL as a symbol instead of empty
\pset linestyle unicode -- pretty box-drawing in output
\pset border 2    -- borders around cells
```

For the cheat sheet you'll want to memorize: `\x auto`, `\timing on`, `\pset null 'Ø'`. Set these in your `~/.psqlrc`:

```
\set QUIET 1
\x auto
\timing on
\pset null 'Ø'
\pset linestyle unicode
\pset border 2
\set HISTFILE ~/.psql_history- :DBNAME
\set HISTCONTROL ignoredups
\set HISTSIZE 5000
\set COMP_KEYWORD_CASE upper
\unset QUIET
```

`HISTFILE :DBNAME` gives you per-database history — your `botdb` history doesn't mix with your `dashdb` history. Quietly transformative.

Variables and scripting

```
\set user_id 42
SELECT * FROM users WHERE id = :user_id;

\set table_name 'users'
SELECT * FROM :table_name;      -- quoted identifier
```

For repeatable scripts, put SQL in a file and `\i` it. For one-offs, type or paste.

Inline command output

`\!` runs a shell command:

```
\! ls -la /tmp
```

Useful for `\! pg_dump ...` or running an external diff against an exported CSV.

Useful one-liners worth memorizing

```
-- Tables in the current DB, sorted by size
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) AS size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC
LIMIT 20;

-- Currently running queries (over 5 seconds)
SELECT pid, age(clock_timestamp(), query_start) AS age, username, query
FROM pg_stat_activity
WHERE state != 'idle' AND query_start < now() - interval '5 seconds'
ORDER BY query_start;

-- Indexes that have never been used (candidates for dropping)
SELECT s.schemaname, s.relname AS table, s.indexrelname AS index,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS size
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan = 0 AND NOT i.indisunique
ORDER BY pg_relation_size(s.indexrelid) DESC;

-- Largest indexes
SELECT schemaname, relname AS table, indexrelname AS index,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 20;

-- Slow queries (requires pg_stat_statements extension)
SELECT round(total_exec_time::numeric, 2) AS total_ms,
       calls,
       round(mean_exec_time::numeric, 2) AS mean_ms,
       substring(query, 1, 80) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

3. Schema design fundamentals

Primary keys

Default: integer surrogate keys (`bigserial` or `bigint GENERATED ALWAYS AS IDENTITY`).

Sequential, dense, fast to compare, fast in indexes. Use for almost everything.

```
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    -- ...
);
```

Use `bigint`, not `int`. `int` is 32-bit and you'll regret it at 2 billion rows (which arrives faster than you'd think for high-volume tables like events, logs). `bigint` is 64-bit and effectively unlimited. The 4 extra bytes per row don't matter.

UUIDs (`uuid` type): when you need globally unique IDs (multi-system event sourcing, distributed inserts, public-facing IDs you don't want to be guessable). Two flavors:

- **v4 (random)** — completely random; bad for index locality (every insert lands in a random page). Use only when locality doesn't matter.
- **v7 (time-ordered, since UUID spec 2024)** — time-prefixed; new inserts cluster, similar to sequential IDs. Generated client-side or via `uuidv7()` if you install a helper extension. Modern default for UUID primary keys.

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";  -- gives uuid_generate_v4() etc.

CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  -- v4, built-in to PG 13+
  -- ...
);
```

Natural keys (e.g., email as the PK of `users`): rarely a good idea. Email changes; foreign keys cascading email changes are painful. Use a surrogate, treat email as a UNIQUE column.

IMPORTANT. DEFAULT TO `BIGINT GENERATED ALWAYS AS IDENTITY`. Reach for UUIDs when you have a specific reason (distributed inserts, public IDs). The "UUIDs everywhere by default" pattern is overcorrection — they're bigger, slower in B-tree indexes, and unnecessary for most personal/web apps.

Foreign keys

```
CREATE TABLE posts (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  body TEXT
);
```

The `ON DELETE` clause matters:

Clause	What it does
<code>ON DELETE CASCADE</code>	Delete child rows when parent is deleted
<code>ON DELETE SET NULL</code>	Null out the FK on parent delete (column must be nullable)
<code>ON DELETE RESTRICT</code>	Prevent parent delete if children exist (default)
<code>ON DELETE NO ACTION</code>	Same as RESTRICT but deferrable

Default is RESTRICT. Pick CASCADE intentionally when the child rows truly belong to the parent (deleting a user deletes their posts). Pick SET NULL when the child rows have meaning beyond the parent (deleting a user doesn't delete their old comments; they become "anonymous").

Always index foreign keys. Postgres doesn't auto-index FK columns; without an index, `ON DELETE CASCADE` and joins on FKs do sequential scans. The exception is when the column is part of a multi-column PK or a unique constraint that already has it indexed.

Normalization

Default: **3NF (third normal form)**. Each fact lives in exactly one place. If you find yourself updating the same value in multiple rows when it changes, that fact belongs in its own table referenced by FK.

When to **denormalize**:

- Reporting / analytics queries that need pre-computed aggregates. Use materialized views (§13) or a separate analytics schema.
- Hot-path read queries where a join is genuinely the bottleneck (rare; the planner is usually good).
- Audit columns (`created_at`, `updated_at`) — always denormalize. Every table gets them.

TIP. EVERY TABLE GETS `CREATED_AT TIMESTAMPTZ NOT NULL DEFAULT NOW()`. Add `updated_at` if the row mutates. These are non-negotiable; you'll thank yourself the first time you need to debug "when did this row appear?"

Naming conventions

Object	Convention
Tables	<code>snake_case</code> , plural (<code>users</code> , <code>posts</code> , <code>order_items</code>)
Columns	<code>snake_case</code> (<code>user_id</code> , <code>created_at</code>)
Primary key	<code>id</code> (always, even when there's a more natural name)
Foreign keys	<code><referenced_table_singular>_id</code> (<code>user_id</code> , <code>post_id</code>)
Indexes	<code>idx_<table>_<columns></code> (<code>idx_posts_user_id</code>)
Unique constraints	<code>uq_<table>_<columns></code>
Check constraints	<code>ck_<table>_<description></code>
Junction (many-to-many) tables	<code><table_a>_<table_b></code> (<code>users_roles</code>)

These aren't enforced. Consistency is the win.

NULL semantics

`NULL` means "unknown," not "empty." Specifically:

- `NULL = NULL` is **NULL** (not TRUE).
- `NULL = anything` is NULL.
- `WHERE x = NULL` returns nothing. Use `WHERE x IS NULL`.
- `COUNT(column)` counts non-NULL values; `COUNT(*)` counts rows.
- Indexes by default include NULLs (you can search for them with `IS NULL`), but in B-tree they sit at the end.

Make columns `NOT NULL` by default. Add nullability only when the absence of a value is genuinely meaningful and different from "we haven't filled it in yet." Mostly: don't use NULL for "unknown" when "" or 0 or a sentinel would do.

Schemas (namespaces)

Postgres has schemas (`public` by default). They group tables logically. Useful for multi-tenant separation, or separating concerns within one DB (`public.users` vs `audit.users_history` vs `archive.users_2023`).

```
CREATE SCHEMA audit;
CREATE TABLE audit.events (...);
SET search_path TO audit, public; -- per-session search path
```

For most personal apps: stick with `public`. Reach for schemas when you have a specific reason (audit, archive, multi-tenancy).

4. Data types: the full menu

The types that matter, with traps.

Numeric types

Type	Range	When to use
<code>smallint</code> (int2)	-32768 to 32767	Tiny enums, age, etc. Rarely worth the savings.
<code>integer</code> (int4)	±2 billion	Default for most ints. Don't use for PKs that grow.
<code>bigint</code> (int8)	±9.2 quintillion	PKs, anything counting. Default.
<code>numeric</code> / <code>decimal</code>	Arbitrary precision	Money, anywhere precision matters.
<code>real</code> (float4)	6 digits precision	Approximate. Scientific.
<code>double precision</code> (float8)	15 digits	Approximate. Most floating-point.
<code>serial</code> / <code>bigserial</code>	Auto-increment	Older syntax; use <code>GENERATED AS IDENTITY</code> instead in PG 10+.

CAUTION. NEVER USE `FLOAT`/`REAL`/`DOUBLE` FOR MONEY OR ANY VALUE WHERE PRECISION MATTERS. Floating-point arithmetic is approximate. `0.1 + 0.2 != 0.3` in floats. Use `numeric(precision, scale)` for money:

```
sql price_cents BIGINT NOT NULL -- best: integer cents -- or price NUMERIC(12, 2) NOT
NULL CHECK (price >= 0) -- decimal with 2 digit cents
```

Text types

Type	Length	When to use
<code>text</code>	Unlimited	Default. Use this.
<code>varchar(n)</code>	Limited to n chars	When you have a business reason for the limit (column max in legacy systems). Otherwise use <code>text</code> .
<code>char(n)</code>	Fixed length	Rarely. Always-padded; almost never the right call.

In Postgres there is **no performance difference** between `text` and `varchar`. The "use varchar for efficiency" advice is legacy from other databases. Use `text` and add a CHECK constraint if you need a length limit:

```
name TEXT NOT NULL CHECK (char_length(name) <= 100)
```

Date and time types

Type	What it stores	Notes
<code>timestamp</code>	Date+time, no TZ	Don't use. Will confuse you.
<code>timestampz</code> (<code>timestamp with time zone</code>)	UTC + display TZ	Default. Always store in UTC; render in user TZ.
<code>date</code>	Date only	When time doesn't matter (birthdays, schedules).
<code>time</code>	Time only	Rare.
<code>interval</code>	Duration	<code>INTERVAL '3 days', '2 hours'</code> .

IMPORTANT. `TIMESTAMPTZ`, ALWAYS. Postgres stores it as UTC; the "with time zone" name is misleading — it just means "I know the timezone and can convert for display." Plain `timestamp` stores wall-clock time without timezone context, which is ambiguous and bug-prone.

```
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
-- now() returns the current transaction start time in UTC

SELECT created_at AT TIME ZONE 'America/Chicago' FROM events;
-- Render in Central Time for display
```

Boolean

```
is_active BOOLEAN NOT NULL DEFAULT true
```

Standard true/false. Postgres accepts `true/false`, `t/f`, `1/0`, `'on'/'off'`, `'yes'/'no'` as input. Use `true/false` in code.

UUID

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Built-in `gen_random_uuid()` is v4 (random). For v7 (time-ordered), use the `uuidv7()` extension or generate client-side.

Binary

```
file_content BYTEA NOT NULL
```

`bytea` for arbitrary bytes. **Don't store large files in Postgres** — use object storage (S3, B2) and store the URL/key as text. Postgres works fine for small binary data (hashes, signatures, small thumbnails); not great for documents.

Arrays

```
tags TEXT[] NOT NULL DEFAULT '{}'  
  
INSERT INTO posts (title, tags) VALUES ('hello', ARRAY['greeting', 'intro']);  
  
SELECT * FROM posts WHERE 'greeting' = ANY(tags);  
SELECT * FROM posts WHERE tags @> ARRAY['greeting']; -- contains  
SELECT * FROM posts WHERE tags && ARRAY['greeting', 'intro']; -- overlaps
```

Index with `GIN`:

```
CREATE INDEX idx_posts_tags ON posts USING GIN (tags);
```

WARNING. DON'T USE ARRAYS FOR MANY-TO-MANY RELATIONSHIPS. They look easy but they're a query maintenance nightmare. Use a junction table.

Arrays are great for:

- Bounded sets you'll never query in complex ways (a row's tags, a user's role names if you don't have a roles table).
- When you'd otherwise serialize a small list into JSON.

JSON / JSONB

See §7 for the deep dive. Short version:

- `jsonb` (always, never plain `json`). Binary representation, supports indexing, slightly slower to write but much faster to query.
- Use for semi-structured data — webhook payloads, flexible attributes, configuration.
- Don't use for primary application data when columns would do.

Enums

Two ways: a real enum type, or a TEXT column with a CHECK.

ENUM type:

```
CREATE TYPE order_status AS ENUM ('pending', 'paid', 'shipped', 'delivered', 'cancelled');  
  
CREATE TABLE orders (  
    id BIGSERIAL PRIMARY KEY,  
    status order_status NOT NULL DEFAULT 'pending'  
);
```

Pros: compact storage; type-checked.

Cons: **adding a value requires a migration.** Removing or reordering is even harder. Painful for evolving applications.

TEXT + CHECK:

```
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  status TEXT NOT NULL DEFAULT 'pending'
  CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled'))
);
```

Pros: easy to add/remove values (just update the CHECK).

Cons: slightly larger storage; less compile-time safety.

Recommendation: TEXT + CHECK for almost everything. The flexibility wins.

Other types worth knowing

- `inet`, `cidr` — IP addresses / networks.
- `macaddr` — MAC addresses.
- `tsvector`, `tsquery` — for full-text search (§8).
- `int4range`, `tstzrange`, etc. — range types. Useful for "valid between these dates" patterns.
- `point`, `circle`, `polygon` — geometric types. For geo work, use PostGIS instead.
- `xml` — exists. Skip; use `jsonb`.
- `money` — exists. Skip; use `numeric` or integer cents.

5. Indexes: every type, when to use

Indexes trade write cost (every INSERT/UPDATE/DELETE updates indexes) for read speed.

Postgres has multiple index types tuned for different query patterns.

B-tree (the default)

```
CREATE INDEX idx_posts_user_id ON posts(user_id);
```

For most things. Equality (`=`), range (`>`, `<`, `BETWEEN`), prefix (`LIKE 'abc%'`), `ORDER BY`, `IS NULL`. The default when you don't specify a type.

Multi-column B-tree:

```
CREATE INDEX idx_posts_user_created ON posts(user_id, created_at DESC);
```

Useful when you frequently query "posts by user X, latest first." Postgres can use the leading prefix of a multi-column index, so this index also serves queries that filter only by `user_id`.

Order matters. Put the most-filtered column first. The trailing columns help only for queries that also filter or sort by the leading ones.

Hash

```
CREATE INDEX idx_sessions_token ON sessions USING HASH (token);
```

Only equality. Smaller than B-tree for exact matches. Was unreliable pre-Postgres 10; now safe to use. Use only when you've benchmarked and confirmed it beats B-tree for your specific case (usually it doesn't).

GIN (Generalized Inverted Index)

```
CREATE INDEX idx_posts_tags ON posts USING GIN (tags);
CREATE INDEX idx_events_data ON events USING GIN (data); -- jsonb
CREATE INDEX idx_articles_search ON articles USING GIN (to_tsvector('english', body));
```

For **container types** — arrays, JSONB, full-text search vectors. GIN indexes the contents of the value, not the value itself.

Larger than B-tree but enables `@>` (contains), `?` (key exists), `?|`, `?&`, and full-text query operators.

GiST (Generalized Search Tree)

```
CREATE INDEX idx_events_range ON events USING GIST (validity tstzrange_ops);
CREATE INDEX idx_locations ON locations USING GIST (coordinates); -- PostGIS
```

For **range types**, **geometric types** (PostGIS), exclusion constraints (§9). Good for "find rows whose range overlaps this range."

BRIN (Block Range Index)

```
CREATE INDEX idx_logs_created_at_brin ON logs USING BRIN (created_at);
```

Tiny index that stores min/max for blocks of rows. Useful for huge tables where the indexed column correlates with physical row order (e.g., `created_at` on a log table that's only ever appended). Trades precision for size — query planner narrows to a range of pages and then scans them.

When BRIN wins: tables with billions of rows, naturally ordered. When it doesn't: small tables, or columns that aren't correlated with insert order.

Partial indexes

```
CREATE INDEX idx_users_active ON users(email) WHERE deleted_at IS NULL;
```

Only indexes rows matching the WHERE clause. Smaller, faster. Used by the planner only when the query has the same predicate.

Use case: "active" filter you always include. Most queries against soft-deleted-row tables filter `WHERE deleted_at IS NULL`. A partial index on that predicate is much smaller than a full index.

Expression indexes

```
CREATE INDEX idx_users_email_lower ON users(lower(email));

-- Then queries that use lower(email) hit the index:
SELECT * FROM users WHERE lower(email) = lower('Joshua@Example.com');
```

Indexes the result of an expression. Postgres can use the index for queries that compute the same expression.

Use case: case-insensitive search, parsing date parts, computed predicates.

Covering indexes (INCLUDE)

```
CREATE INDEX idx_orders_user_id ON orders(user_id) INCLUDE (status, total);
```

Stores extra non-key columns in the index leaf. Allows "index-only scans" — Postgres can answer the query from the index alone, without touching the table heap.

Use when you have a frequent query like `SELECT status, total FROM orders WHERE user_id = X` and want it to avoid the table.

Unique indexes

```
CREATE UNIQUE INDEX uq_users_email ON users(email);

-- Or as a constraint (creates an index implicitly):
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);
```

Enforces uniqueness; usable as a B-tree index. Functionally identical to a UNIQUE constraint — the constraint syntax is preferred for clarity.

Concurrent index creation

```
CREATE INDEX CONCURRENTLY idx_huge_table_x ON huge_table(x);
```

Builds the index without locking the table for writes. Slower than a regular `CREATE INDEX`, but doesn't block your application. **Always use CONCURRENTLY on production tables** unless the table is tiny.

Caveats:

- Can't run inside a transaction.
- Fails leave a "INVALID" index behind that you must drop manually.
- Doesn't help with the initial write cost — uses extra IO, slower than `CREATE INDEX`.

When to add an index

1. You have a query that runs frequently.
2. `EXPLAIN` shows a sequential scan or a slow index.
3. The table is large enough that the scan is noticeable.
4. The column you're filtering on has good selectivity (rare values).

When NOT to add an index

- Small tables (under ~10k rows). Sequential scan is faster than index lookup overhead.
- Columns with low cardinality (boolean, status with few values) — unless you need a partial index.
- "Just in case." Indexes cost INSERT/UPDATE performance. Add them in response to real queries.

Finding unused indexes

```
SELECT s.schemaname, s.relname AS table, s.indexrelname AS index,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS size,
       s.idx_scan AS scans
FROM pg_stat_user_indexes s
JOIN pg_index i ON i.indexrelid = s.indexrelid
WHERE s.idx_scan < 50 AND NOT i.indisunique AND NOT i.indisprimary
ORDER BY pg_relation_size(s.indexrelid) DESC;
```

Indexes with very low `idx_scan` counts after a representative amount of time are candidates for dropping. Drop with `DROP INDEX CONCURRENTLY <name>;`.

6. Query patterns worth knowing

The SQL features that turn "I can do this with three queries and a loop" into "I can do this in one query."

Common Table Expressions (CTEs)

```
WITH recent_orders AS (
    SELECT * FROM orders WHERE created_at > now() - interval '7 days'
),
order_totals AS (
    SELECT user_id, sum(total) AS week_total
    FROM recent_orders
    GROUP BY user_id
)
SELECT u.name, ot.week_total
FROM order_totals ot
JOIN users u ON u.id = ot.user_id
ORDER BY ot.week_total DESC;
```

CTEs let you name intermediate results. They make complex queries readable.

NOTE. PRE-POSTGRES 12, CTES WERE ALWAYS MATERIALIZED (treated as optimization fences). Postgres 12+ inlines them by default, like subqueries. Use the `MATERIALIZED` keyword if you specifically want materialization (e.g., a CTE referenced multiple times in a complex query).

Recursive CTEs

For tree/graph traversal:

```

WITH RECURSIVE comment_tree AS (
  -- Anchor: top-level comments
  SELECT id, parent_id, body, 0 AS depth
  FROM comments
  WHERE parent_id IS NULL AND post_id = 42

  UNION ALL

  -- Recursion: children of already-found comments
  SELECT c.id, c.parent_id, c.body, ct.depth + 1
  FROM comments c
  JOIN comment_tree ct ON c.parent_id = ct.id
)
SELECT * FROM comment_tree ORDER BY depth, id;

```

Use for: comment threads, org charts, file system paths, dependency trees.

Window functions

```

SELECT user_id, created_at, total,
       row_number() OVER (PARTITION BY user_id ORDER BY created_at DESC) AS recency,
       sum(total) OVER (PARTITION BY user_id) AS user_total
FROM orders;

```

`PARTITION BY` is like `GROUP BY` but doesn't collapse rows. Each row gets the aggregate computed over its partition.

Common windows: - `row_number()` — sequential numbering within partition - `rank()`, `dense_rank()` — ranking with/without gaps - `lag(col, n)`, `lead(col, n)` — previous/next row values - `first_value()`, `last_value()`, `nth_value()` — specific position - `sum()`, `avg()`, `count()`, etc. — running totals when paired with frames

Frame specification (for running totals):

```

SELECT created_at, amount,
       sum(amount) OVER (
         ORDER BY created_at
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM transactions;

```

`DISTINCT ON`

"Give me the latest row per user":

```

SELECT DISTINCT ON (user_id) user_id, created_at, status
FROM orders
ORDER BY user_id, created_at DESC;

```

Postgres-specific. Faster and clearer than the equivalent window-function approach for "one row per group."

`LATERAL` joins

```
SELECT u.name, p.title, p.created_at
FROM users u
CROSS JOIN LATERAL (
  SELECT title, created_at
  FROM posts
  WHERE user_id = u.id
  ORDER BY created_at DESC
  LIMIT 3
) p;
```

The subquery in the `LATERAL` block can reference columns from the preceding rows. Useful for "top N per group" patterns and complex correlated subqueries.

`UPSERT` (`INSERT ... ON CONFLICT`)

```
INSERT INTO users (email, name)
VALUES ('alice@example.com', 'Alice')
ON CONFLICT (email) DO UPDATE
  SET name = EXCLUDED.name,
      updated_at = now();

-- Or insert-or-skip:
INSERT INTO users (email, name)
VALUES ('alice@example.com', 'Alice')
ON CONFLICT (email) DO NOTHING;
```

`EXCLUDED` refers to the row that would have been inserted. The `ON CONFLICT` clause requires a unique constraint or index on the conflict target.

`RETURNING`

Any `INSERT`, `UPDATE`, or `DELETE` can return the affected rows:

```
INSERT INTO users (email, name) VALUES ('a@b.com', 'A')
RETURNING id, created_at;

UPDATE orders SET status = 'shipped' WHERE id = 5
RETURNING *;

DELETE FROM sessions WHERE expires_at < now()
RETURNING id;
```

Avoids the "insert + select to get the ID" round trip. Always use it when you need IDs or computed values after a write.

Set-returning functions in `SELECT`

```
SELECT generate_series(1, 12) AS month;
SELECT unnest(ARRAY['a', 'b', 'c']);
```

`generate_series` and `unnest` are invaluable for synthetic data, gap-filling, expanding arrays.

`FILTER` for conditional aggregates

```
SELECT
  count(*) AS total,
  count(*) FILTER (WHERE status = 'paid') AS paid_count,
  count(*) FILTER (WHERE status = 'cancelled') AS cancelled_count,
  sum(total) FILTER (WHERE status = 'paid') AS paid_revenue
FROM orders;
```

Cleaner than `COUNT(CASE WHEN ... THEN 1 END)`.

Multi-row `VALUES`

```
-- Use as a table source
SELECT v.id, v.name
FROM (VALUES (1, 'a'), (2, 'b'), (3, 'c')) AS v(id, name);

-- Bulk insert
INSERT INTO things (id, name) VALUES
  (1, 'a'),
  (2, 'b'),
  (3, 'c');
```

7. JSON / JSONB deep dive

`jsonb` is for semi-structured data — webhook payloads, flexible attributes, configuration that varies by row, anywhere you'd otherwise add many nullable columns.

`json` vs `jsonb`

	<code>json</code>	<code>jsonb</code>
Storage	Text, exact representation	Binary, parsed
Whitespace preserved	Yes	No
Key order preserved	Yes	No
Duplicate keys	Allowed	Last wins
Indexing	Limited	GIN, GIST
Query speed	Reparses each access	Fast

Always use `jsonb`. The reasons to use `json` instead are vanishingly rare (you need round-trip exactness for some legacy reason).

Basic operations

```
CREATE TABLE events (
  id BIGSERIAL PRIMARY KEY,
  payload JSONB NOT NULL DEFAULT '{}')
);

INSERT INTO events (payload) VALUES
  ('{"user_id": 1, "action": "login", "ip": "192.168.1.1"}'),
  ('{"user_id": 2, "action": "click", "url": "/page"}');

-- Access fields
SELECT payload->'user_id' FROM events;           -- as jsonb
SELECT payload->>'user_id' FROM events;           -- as text
SELECT (payload->>'user_id')::int FROM events; -- cast to int

-- Nested access
SELECT payload->'meta'->>'browser' FROM events;
SELECT payload#>>'{meta, browser}' FROM events; -- path-based, same result

-- Existence
SELECT * FROM events WHERE payload ? 'user_id';           -- key exists
SELECT * FROM events WHERE payload ?| ARRAY['user_id', 'name']; -- any
SELECT * FROM events WHERE payload ?& ARRAY['user_id', 'name']; -- all

-- Contains
SELECT * FROM events WHERE payload @> '{"action": "login"}';
-- Returns events whose payload contains all the fields in the RHS

-- Contained by (the reverse)
SELECT * FROM events WHERE '{"user_id": 1}' <@ payload;
```

Updating JSONB

```
-- Set a field (creates if missing, overwrites if present)
UPDATE events SET payload = payload || '{"processed": true}' WHERE id = 1;

-- jsonb_set: set a nested field
UPDATE events SET payload = jsonb_set(payload, '{meta, browser}', '"Firefox"')
WHERE id = 1;

-- Remove a key
UPDATE events SET payload = payload - 'temp' WHERE id = 1;

-- Remove a nested path
UPDATE events SET payload = payload #- '{meta, ip}' WHERE id = 1;
```

Indexing JSONB

GIN with default `jsonb_ops` — supports `?`, `?|`, `?&`, `@>`, `@?`, `@@`:

```
CREATE INDEX idx_events_payload ON events USING GIN (payload);
```

GIN with `jsonb_path_ops` — only supports `@>` but smaller and faster for that operator:

```
CREATE INDEX idx_events_payload_path ON events USING GIN (payload jsonb_path_ops);
```

Expression index for a specific key — fastest for queries on one known field:

```
CREATE INDEX idx_events_user_id ON events ((payload->>'user_id'));  
  
SELECT * FROM events WHERE payload->>'user_id' = '1'; -- uses the index
```

JSONPath

Postgres 12+ supports SQL/JSON path expressions:

```
SELECT * FROM events WHERE payload @? '$.user_id ? (@ == 1)';  
SELECT jsonb_path_query(payload, '$.meta.browser') FROM events;
```

Powerful for complex extractions. The syntax is a bit fiddly; reach for it when simpler operators don't fit.

When to use JSONB vs columns

JSONB: - Schema is genuinely variable (webhook payloads from many sources). - Sparse attributes (each row has a few of many possible fields). - You'll search the data but rarely filter on most fields. - Temporary/exploratory schema before promoting fields to columns.

Columns: - Same fields on every row. - Fields you'll join on, group by, or aggregate frequently. - Need strong typing or NOT NULL constraints.

Mixed pattern (common):

```
CREATE TABLE events (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL REFERENCES users(id),  
  event_type TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  payload JSONB NOT NULL DEFAULT '{}' -- everything event-specific  
);
```

Promote a field from JSONB to a column the moment you find yourself filtering on it heavily.

8. Full-text search

Postgres has built-in full-text search that handles most needs short of "I'm building a search engine."

The basics

```
-- Create a tsvector (the searchable representation)
SELECT to_tsvector('english', 'The quick brown fox jumps over the lazy dog');
-- 'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2

-- Create a tsquery (the query)
SELECT to_tsquery('english', 'fox & dog');
-- 'fox' & 'dog'

-- Match
SELECT to_tsvector('english', 'fox and dog') @@ to_tsquery('english', 'fox & dog');
-- true
```

Note that `to_tsvector` does **stemming** (`jumps` → `jump`), **stop word removal** (`the`, `over` dropped), and **lowercasing**. The query goes through the same normalization. So "running" matches "ran" matches "runs."

A practical setup

For an `articles` table with searchable `title` and `body`:

```
ALTER TABLE articles ADD COLUMN search_vector tsvector
    GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
        setweight(to_tsvector('english', coalesce(body, '')), 'B')
    ) STORED;

CREATE INDEX idx_articles_search ON articles USING GIN (search_vector);

-- Search
SELECT id, title, ts_rank(search_vector, query) AS rank
FROM articles, to_tsquery('english', 'postgres & deep:*') AS query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 10;
```

What's happening:

- **GENERATED ALWAYS AS ... STORED** — the search vector auto-updates when title/body change. Older guides use triggers; generated columns are cleaner.
- **setweight('A')** and **setweight('B')** — title matches rank higher than body matches.
- **coalesce(..., '')** — handle NULL fields gracefully.
- **ts_rank()** — score how well a row matches the query.
- **deep:*** — prefix match (matches "deep", "deeply", "deepening").

Query syntax

```
to_tsquery('english', 'foo & bar')      -- both
to_tsquery('english', 'foo | bar')      -- either
to_tsquery('english', 'foo & !bar')     -- foo AND NOT bar
to_tsquery('english', 'foo <-> bar')     -- foo followed immediately by bar (phrase)
to_tsquery('english', 'foo <2> bar')    -- foo, then bar within 2 words
to_tsquery('english', 'foo:*')          -- prefix match
```

For user-input search (where users type natural strings), use `websearch_to_tsquery` — it accepts a more forgiving syntax:

```
SELECT * FROM articles WHERE search_vector @@ websearch_to_tsquery('english', 'postgres "deep dive" -mysql');
```

That handles quoted phrases, `-` for exclusion, and free-form OR.

Dictionaries / configurations

`'english'` is the default text search configuration; it includes English stop words and a stemmer. List all available:

```
\dF
```

Custom configurations exist (multilingual, no stop words, etc.). For most personal use, `english` is fine.

When to graduate

Postgres FTS handles probably 90% of search needs at scale up to ~10M rows. Beyond that, or when you need:

- **Faceted search** with aggregations
- **Fuzzy matching** at scale
- **Advanced ranking** with ML signals
- **Cross-table search** with custom scoring

...look at **OpenSearch** / **Elasticsearch** / **MeiliSearch** / **Typesense**. But start with Postgres FTS; many projects never outgrow it.

Trigram search (fuzzy match)

For "find rows where this column is approximately X":

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE INDEX idx_users_name_trgm ON users USING GIN (name gin_trgm_ops);

-- Find names similar to "joshua"
SELECT name, similarity(name, 'joshua') AS sim
FROM users
WHERE name % 'joshua'
ORDER BY sim DESC
LIMIT 10;

-- Or with the operator class:
SELECT * FROM users WHERE name ILIKE '%joshu%'; -- uses the trigram index too
```

`pg_trgm` is excellent for "find user by partial/misspelled name," autocomplete with typo tolerance, and free-text-with-typos search. Complements FTS.

9. Constraints beyond NOT NULL

CHECK constraints

```
CREATE TABLE products (  
  id BIGSERIAL PRIMARY KEY,  
  price_cents BIGINT NOT NULL CHECK (price_cents > 0),  
  discount_pct INTEGER CHECK (discount_pct BETWEEN 0 AND 100),  
  status TEXT NOT NULL CHECK (status IN ('active', 'discontinued', 'draft'))  
);
```

Enforced on every insert/update. Useful for:

- Value ranges (`age BETWEEN 0 AND 150`).
- Format requirements (`email LIKE '%@%'`).
- Status enums.
- Mutually exclusive fields (`CHECK (NOT (foo IS NOT NULL AND bar IS NOT NULL))`).

Don't go overboard — application code can also enforce these. Add CHECKs for invariants that should never be violated, regardless of code bugs.

UNIQUE constraints

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  email TEXT NOT NULL UNIQUE,  
  UNIQUE (twitter_handle)  
);  
  
-- Composite uniqueness:  
ALTER TABLE memberships ADD CONSTRAINT uq_user_org UNIQUE (user_id, org_id);
```

Creates an implicit unique B-tree index. Use for natural uniqueness (email, handle, slug).

Partial uniqueness via a unique partial index:

```
-- Only one "active" subscription per user  
CREATE UNIQUE INDEX uq_active_subscription  
  ON subscriptions(user_id)  
  WHERE status = 'active';
```

This is one of the cleanest patterns for "soft uniqueness."

EXCLUSION constraints

Generalizes uniqueness — "no two rows have overlapping ranges" / "no two rooms booked at overlapping times":

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE room_bookings (
    id BIGSERIAL PRIMARY KEY,
    room_id BIGINT NOT NULL,
    during TSTZRANGE NOT NULL,
    EXCLUDE USING GIST (room_id WITH =, during WITH &&)
);

-- Now you can't insert two bookings for the same room with overlapping times.
INSERT INTO room_bookings (room_id, during) VALUES
    (1, '[2026-06-01 10:00, 2026-06-01 11:00)'),
    (1, '[2026-06-01 10:30, 2026-06-01 11:30)'); -- ERROR: conflicts
```

Use for scheduling, resource booking, allocation slots — anywhere "no overlapping ranges" is the rule.

GENERATED columns

```
CREATE TABLE invoices (
    id BIGSERIAL PRIMARY KEY,
    subtotal_cents BIGINT NOT NULL,
    tax_cents BIGINT NOT NULL,
    total_cents BIGINT GENERATED ALWAYS AS (subtotal_cents + tax_cents) STORED
);
```

The column is computed from other columns and stored. Use for:

- Derived values you always want available (sums, normalized text, tsvectors).
- Avoiding application-side bugs where the computed value drifts from inputs.

Postgres also supports `GENERATED ALWAYS AS ... VIRTUAL`, but as of 16 it's not implemented — only STORED is.

Deferrable constraints

```
CREATE TABLE t (
    id INT PRIMARY KEY DEFERRABLE INITIALLY DEFERRED
);
```

Allows the constraint to be violated mid-transaction and checked at commit. Useful for cycle-resolving migrations (e.g., circular FKs). Rare in personal projects.

10. Transactions and isolation

Basics

```
BEGIN;  
    UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
    UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;  
-- Or: ROLLBACK;
```

Postgres is by default in autocommit mode (each statement is its own transaction). `BEGIN` opens an explicit transaction. `COMMIT` or `ROLLBACK` ends it.

Isolation levels

Four standard levels, Postgres implements three meaningfully:

Level	What it prevents	Postgres impl
READ UNCOMMITTED	(nothing in PG)	Same as READ COMMITTED
READ COMMITTED	Dirty reads	Default
REPEATABLE READ	Dirty + non-repeatable reads	Snapshot isolation
SERIALIZABLE	All anomalies (true serializability)	SSI (Serializable Snapshot Isolation)

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
-- ... your queries ...  
COMMIT;
```

Or for the whole session:

```
SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

When to use what

- **READ COMMITTED (default)** — fine for most operations. Each statement sees a snapshot that includes everything committed before it started. Different statements in the same transaction may see different snapshots.
- **REPEATABLE READ** — when a transaction has multiple statements that need a consistent view of the data. E.g., a complex report that queries multiple tables and expects them to be consistent with each other.
- **SERIALIZABLE** — when you have concurrent transactions that read-then-write based on what they read, and you need the result to be as if they ran one-at-a-time. Higher overhead; some transactions may get serialization failures and need retry. Good for financial logic and anything where correctness > throughput.

IMPORTANT. USE SERIALIZABLE MORE THAN YOU THINK. For personal apps with low concurrency, the cost is negligible. The correctness guarantee is enormous. Postgres's SSI is one of the better implementations; retries on serialization failure are rare. Wrap critical operations (money transfers, atomic increments where you compute the value app-side, etc.) in `SERIALIZABLE` transactions.

Savepoints (nested transactions)

```
BEGIN;
  INSERT INTO accounts (id, balance) VALUES (1, 100);
  SAVEPOINT before_risky;
  INSERT INTO accounts (id, balance) VALUES (1, 200);  -- duplicate key error
  ROLLBACK TO before_risky;
  -- the first INSERT survived
COMMIT;
```

Useful in long transactions where part of the work might need to be undone. Rarely needed in application code (use multiple smaller transactions instead).

11. Locks: explicit and implicit

Postgres locks at multiple granularities:

- **Row locks** — implicit from `UPDATE`/`DELETE`/`SELECT FOR UPDATE`.
- **Table locks** — implicit from DDL (`ALTER TABLE`, etc.) and explicit via `LOCK TABLE`.
- **Page locks** — internal, you don't see these.
- **Advisory locks** — application-level, you opt in.

Row-level locking

```
-- Reserve rows for update; other transactions wait
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;

-- Don't wait; fail if already locked
SELECT * FROM accounts WHERE id = 1 FOR UPDATE NOWAIT;

-- Don't wait; skip locked rows (job-queue pattern)
SELECT * FROM jobs WHERE status = 'pending' FOR UPDATE SKIP LOCKED LIMIT 1;
```

`FOR UPDATE SKIP LOCKED` is the canonical "I'm a worker grabbing the next job from a queue table" pattern. Multiple workers can run the same query and each get a different unlocked row.

Table-level locking

Most DDL operations take strong locks:

DDL	Lock taken
<code>ALTER TABLE ADD COLUMN ...</code> (with no default)	ACCESS EXCLUSIVE (briefly)
<code>ALTER TABLE ADD COLUMN ... DEFAULT ...</code> (PG 11+)	ACCESS EXCLUSIVE (briefly, no rewrite)
<code>CREATE INDEX ...</code>	SHARE (blocks writes)
<code>CREATE INDEX CONCURRENTLY ...</code>	SHARE UPDATE EXCLUSIVE (blocks DDL only)
<code>DROP INDEX ...</code>	ACCESS EXCLUSIVE
<code>DROP INDEX CONCURRENTLY ...</code>	SHARE UPDATE EXCLUSIVE
<code>TRUNCATE TABLE ...</code>	ACCESS EXCLUSIVE
<code>VACUUM</code>	SHARE UPDATE EXCLUSIVE
<code>VACUUM FULL</code>	ACCESS EXCLUSIVE (full table rewrite)

WARNING. `VACUUM FULL` **REWRITES THE ENTIRE TABLE**. Locks it for writes and reads for the duration. Don't run on a live production table unless you've planned downtime. Use `pg_repack` extension for online table rewrites.

Deadlocks

Deadlocks happen when two transactions hold locks the other needs:

```
Tx A: UPDATE accounts WHERE id=1; (holds row 1)
Tx B: UPDATE accounts WHERE id=2; (holds row 2)
Tx A: UPDATE accounts WHERE id=2; (waits for B)
Tx B: UPDATE accounts WHERE id=1; (waits for A) → deadlock
```

Postgres detects deadlocks and aborts one transaction with `ERROR: deadlock detected`. To avoid them, always acquire locks in the same order.

For multi-row UPDATES, order by primary key:

```
UPDATE accounts SET balance = balance + 100
WHERE id IN (1, 2, 3)
ORDER BY id;    -- consistent order across transactions
```

(`UPDATE` doesn't actually accept `ORDER BY` — you'd need a CTE if order truly matters: `WITH x AS (SELECT id FROM accounts WHERE ... ORDER BY id FOR UPDATE) UPDATE ... FROM x ...`. But locks within a single statement are typically taken in the planner's chosen order; consistency across statements is the goal.)

Advisory locks

App-level locks Postgres tracks but doesn't enforce on data:

```
SELECT pg_advisory_lock(42);    -- block until lock acquired
-- ... critical section ...
SELECT pg_advisory_unlock(42);
```

Useful as a distributed mutex without a separate lock service. Common pattern: lock by hash of a resource name (`pg_advisory_lock(hashtext('user-42-import'))`).

12. Triggers and PL/pgSQL

Triggers run code automatically on INSERT/UPDATE/DELETE. Useful but easy to overuse.

When triggers are appropriate

- **Audit logging** — write a row to an audit table on every change.
- **Maintaining derived data** — update a counter, refresh a search vector (though `GENERATED` columns are usually cleaner).
- **Enforcing complex invariants** — when CHECK isn't enough.

When NOT to use triggers

- Business logic — keep it in the app where it's testable and debuggable.
- Anything users will need to understand without reading the DB schema.
- Cascades that could be FKs instead.

A simple trigger

```
-- Function that the trigger calls
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger that calls it
CREATE TRIGGER trg_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW
    EXECUTE FUNCTION set_updated_at();
```

Now every UPDATE on `users` automatically updates `updated_at`.

Trigger types

Type	When	Common use
<code>BEFORE INSERT/UPDATE</code>	Before the row is written	Modify <code>NEW</code> , validate
<code>AFTER INSERT/UPDATE/DELETE</code>	After the row is written	Audit log, notify
<code>INSTEAD OF</code> (on views)	Replaces the operation	Updatable views
<code>FOR EACH ROW</code>	Once per affected row	Most cases
<code>FOR EACH STATEMENT</code>	Once per statement	Bulk operations

Audit pattern

```
CREATE TABLE audit_log (  
    id BIGSERIAL PRIMARY KEY,  
    table_name TEXT NOT NULL,  
    row_id BIGINT NOT NULL,  
    op TEXT NOT NULL,  
    old_data JSONB,  
    new_data JSONB,  
    changed_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
    changed_by TEXT  
);  
  
CREATE OR REPLACE FUNCTION audit_row_changes()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO audit_log (table_name, row_id, op, old_data, new_data, changed_by)  
    VALUES (  
        TG_TABLE_NAME,  
        COALESCE(NEW.id, OLD.id),  
        TG_OP,  
        CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN to_jsonb(OLD) END,  
        CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN to_jsonb(NEW) END,  
        current_setting('app.current_user', true)  
    );  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_users_audit  
AFTER INSERT OR UPDATE OR DELETE ON users  
FOR EACH ROW  
EXECUTE FUNCTION audit_row_changes();
```

`TG_OP`, `TG_TABLE_NAME`, etc. are special trigger variables.

`current_setting('app.current_user', true)` reads a custom session variable that the app sets before queries: `SELECT set_config('app.current_user', 'joshua', false);`. This gets the application user (not the DB user) into the audit log.

PL/pgSQL basics

```
DO $$  
DECLARE  
    user_count INT;  
BEGIN  
    SELECT count(*) INTO user_count FROM users;  
    RAISE NOTICE 'There are % users', user_count;  
END;  
$$;
```

`DO` is a one-off anonymous block. For reusable logic, create a function:

```

CREATE OR REPLACE FUNCTION calculate_score(user_id BIGINT)
RETURNS INT AS $$
DECLARE
    base_score INT;
    bonus_score INT;
BEGIN
    SELECT COALESCE(sum(value), 0) INTO base_score
    FROM scores WHERE user_id = $1;

    SELECT count(*) INTO bonus_score
    FROM achievements WHERE user_id = $1;

    RETURN base_score + (bonus_score * 10);
END;
$$ LANGUAGE plpgsql STABLE;

```

`STABLE` (or `IMMUTABLE` or `VOLATILE`) is a hint about determinism — affects planning and parallelization. `STABLE` means "same input → same output within a single query."

Other languages: PL/Python, PL/V8 (JavaScript), PL/Perl. PL/pgSQL is the most common and has the best tooling.

13. Views and materialized views

Regular views

```

CREATE VIEW active_users AS
SELECT * FROM users WHERE deleted_at IS NULL;

SELECT * FROM active_users WHERE created_at > '2026-01-01';

```

A view is a saved query. It's computed every time you query it. Good for:

- Hiding complexity (a complex JOIN behind a simple name).
- Providing a stable interface as underlying tables change.
- Permission boundaries (grant SELECT on the view, not the table).

Materialized views

```

CREATE MATERIALIZED VIEW daily_revenue AS
SELECT date_trunc('day', created_at) AS day,
       sum(total) AS revenue,
       count(*) AS order_count
FROM orders
GROUP BY 1;

REFRESH MATERIALIZED VIEW daily_revenue;
-- Optionally: REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue;
-- (Concurrent refresh requires a unique index on the materialized view.)

```

Materialized views store the result of the query. They're like a cache. Updated by `REFRESH`, not automatically.

When to use:

- Aggregations on huge tables, queried often.
- Pre-computed reports.
- Search indexes built from multiple tables.

Schedule REFRESH via cron or trigger:

```
# Refresh nightly at 3am
0 3 * * * psql -c 'REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue'
```

Updatable views

Views over a single table can be updated:

```
CREATE VIEW active_users AS
SELECT * FROM users WHERE deleted_at IS NULL;

UPDATE active_users SET name = 'new name' WHERE id = 5;
-- works; modifies the underlying users table
```

Views with joins, aggregations, or DISTINCT aren't updatable directly — but you can use an `INSTEAD OF` trigger to make them so. Rarely worth it.

14. `EXPLAIN`: reading query plans

`EXPLAIN` shows you what Postgres plans to do; `EXPLAIN ANALYZE` runs the query and shows actuals.

```
EXPLAIN SELECT * FROM users WHERE email = 'a@b.com';
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'a@b.com';
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT * FROM users WHERE email = 'a@b.com';
```

`BUFFERS` shows page reads (cache hits vs disk reads). `FORMAT JSON` makes it easier to feed into tools.

CAUTION. `EXPLAIN ANALYZE` **ACTUALLY RUNS THE QUERY.** Including `UPDATE`s and `DELETE`s. Wrap in `BEGIN; ... ROLLBACK;` if you don't want side effects.

Reading a plan

```
QUERY PLAN
-----
Index Scan using idx_users_email on users (cost=0.28..8.30 rows=1 width=64)
  Index Cond: (email = 'a@b.com'::text)
```

- **Operation** — `Index Scan`, `Seq Scan`, `Hash Join`, `Nested Loop`, `Merge Join`, `Sort`, `Aggregate`, `Limit`, etc.
- **Target** — what it's reading.

- **Cost** — planner's estimate of work. `(cost=startup..total rows=N width=B)`. Startup is "before the first row," total is "until the last row." Rows is estimated row count. Width is row size in bytes.
- **Index Cond / Filter** — the predicates pushed into the operation.

What to look for

Sign	What it means	What to do
<code>Seq Scan</code> on a large table	Reading every row	Add an index on the filter column
<code>Index Scan</code> then <code>Filter</code> on additional predicates	Index found candidates, filter narrowed	If the filter is selective, consider a composite or partial index
Big difference between <code>estimated rows</code> and <code>actual rows</code>	Statistics are out of date	<code>ANALYZE <table>;</code>
<code>Sort</code> operations	Postgres is reordering for ORDER BY or JOIN	Index supporting the sort order may help
<code>Hash Join</code> with high cost	Building a hash of the smaller table	Usually fine; investigate if memory pressure
<code>Nested Loop</code> over many rows	Iterating one side per row of the other	Likely a missing index on the inner side

Buffer info

With `BUFFERS`:

QUERY PLAN

Seq Scan on orders (cost=0.00..18334.00 rows=1000000 width=44)
Buffers: shared hit=8500 read=2 dirtied=0

- **shared hit** — pages found in cache (fast).
- **read** — pages read from disk (slow).
- **dirtied** — pages modified during read.

High `read` relative to `hit` = cold cache; might just need to be run a second time.

15. Performance: statistics, autovacuum, slow queries

Statistics

Postgres uses table statistics to estimate row counts and choose plans. They're collected by `ANALYZE` (and automatically by autovacuum).

If queries get slow after big data changes, run:

<code>ANALYZE users;</code>	<code>-- analyze one table</code>
<code>ANALYZE;</code>	<code>-- all tables</code>
<code>VACUUM ANALYZE users;</code>	<code>-- combine vacuum + analyze</code>

For specific columns where statistics are critical:

```
ALTER TABLE users ALTER COLUMN status SET STATISTICS 1000;
ANALYZE users;
```

Default `statistics_target` is 100; bumping to 1000 collects finer distribution info at the cost of larger statistics tables.

Autovacuum

Background process that:

1. **Removes dead rows** (from MVCC's UPDATE/DELETE leftovers).
2. **Updates statistics.**
3. **Prevents transaction ID wraparound.**

For most workloads, defaults are fine. For write-heavy tables:

```
ALTER TABLE events SET (
    autovacuum_vacuum_scale_factor = 0.05,      -- vacuum after 5% of table changes (default 20%)
    autovacuum_analyze_scale_factor = 0.02      -- analyze after 2% changes
);
```

Check autovacuum activity:

```
SELECT relname,
       last_autovacuum, last_vacuum,
       last_autoanalyze, last_analyze,
       n_dead_tup, n_live_tup,
       round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

A `dead_pct` above ~20% on a frequently-queried table = autovacuum isn't keeping up.

Slow query logging

In `postgresql.conf`:

```
log_min_duration_statement = 1000    # log statements taking >1s
```

Slow queries appear in the Postgres log. Aggregate analysis is easier with `pg_stat_statements`.

pg_stat_statements

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
-- And in postgresql.conf:
-- shared_preload_libraries = 'pg_stat_statements'
-- (Requires restart)
```

Now this query gives you a sortable list of expensive queries:

```

SELECT
    round(total_exec_time::numeric, 2) AS total_ms,
    calls,
    round(mean_exec_time::numeric, 2) AS mean_ms,
    round((100 * total_exec_time / sum(total_exec_time) OVER ()):numeric, 1) AS pct_total,
    substring(query, 1, 100) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;

```

Reset stats periodically: `SELECT pg_stat_statements_reset();`.

16. Extensions worth knowing

```

SELECT * FROM pg_available_extensions ORDER BY name;
-- Install one:
CREATE EXTENSION IF NOT EXISTS pg_trgm;

```

Extension	What it does	When
<code>pg_trgm</code>	Trigram similarity / fuzzy search	Autocomplete, "find by approximate name"
<code>pgcrypto</code>	Cryptographic functions	Hashing, encryption helpers
<code>uuid-oss</code>	UUID generators (v1, v3, v4, v5)	When <code>gen_random_uuid</code> isn't enough
<code>hstore</code>	Key-value type (jsonb predecessor)	Mostly legacy now; jsonb is better
<code>pg_stat_statements</code>	Query performance stats	Always.
<code>btree_gist</code>	B-tree-like ops via GiST	Required for some exclusion constraints
<code>citext</code>	Case-insensitive text	Email columns mostly
<code>pgvector</code>	Vector similarity for embeddings	RAG / ML on top of Postgres
<code>timescaledb</code>	Time-series optimizations	Time-series at scale, hypertables
<code>postgis</code>	Geographic / geometric	Maps, locations, geofences
<code>pg_partman</code>	Partitioning automation	Large tables; partition by time
<code>pg_repack</code>	Online table rewrite	VACUUM FULL alternative
<code>pgaudit</code>	Audit logging	Compliance-heavy environments

pgvector deep-dive (for ML embeddings)

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY,
  content TEXT NOT NULL,
  embedding VECTOR(1536) NOT NULL    -- OpenAI's text-embedding-3-small
);

CREATE INDEX ON documents USING HNSW (embedding vector_cosine_ops);

-- Find nearest neighbors:
SELECT id, content
FROM documents
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector
LIMIT 10;
```

Operators: `<->` (L2 / Euclidean), `<=>` (cosine), `<#>` (inner product, negated).

Postgres + pgvector handles RAG (retrieval-augmented generation) workloads up to single-machine scale (10s of millions of vectors). At hundreds of millions, look at dedicated vector DBs.

17. `postgresql.conf` knobs that matter

Most defaults are sane. A few are worth knowing:

Memory

Setting	What	Tune
<code>shared_buffers</code>	Postgres's main cache	25% of RAM (default is small)
<code>effective_cache_size</code>	Planner hint about OS cache	50-75% of RAM
<code>work_mem</code>	Per-query operation memory (sort, hash)	16-64 MB; higher = more memory per operation
<code>maintenance_work_mem</code>	For VACUUM, CREATE INDEX	256 MB - 1 GB

Durability vs speed

Setting	What	Tune
<code>synchronous_commit</code>	Wait for WAL flush before COMMIT returns	<code>on</code> (default). Don't change unless you understand the cost.
<code>wal_compression</code>	Compress WAL records	<code>on</code> for write-heavy
<code>checkpoint_completion_target</code>	Spread checkpoint IO over time	0.9 (default)

Connections

Setting	What	Tune
<code>max_connections</code>	Maximum concurrent connections	100 default; lower is often better (use a pooler)

Logging

Setting	What	Tune
<code>log_min_duration_statement</code>	Log slow queries	1000 (1s) is a good starting point
<code>log_checkpoints</code>	Log each checkpoint	<code>on</code>
<code>log_lock_waits</code>	Log waits over <code>deadlock_timeout</code>	<code>on</code>
<code>log_line_prefix</code>	Format of log lines	<code>'%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '</code>

Apply changes

```
-- View current
SHOW shared_buffers;
SHOW work_mem;

-- Set for the session (immediate)
SET work_mem = '64MB';

-- Set for the database (requires reconnect)
ALTER DATABASE mydb SET work_mem = '64MB';

-- Set in postgresql.conf, then:
SELECT pg_reload_conf();    -- for most settings
-- Or restart for settings marked "requires restart" (shared_buffers, max_connections, etc.)
```

18. Connection pooling: pgbouncer

Postgres backends are heavy (one OS process per connection, each using memory). Opening/closing connections is expensive. For web apps with many short-lived requests, **always use a connection pooler**.

pgbouncer is the standard. Runs as a sidecar; apps connect to it; it maintains a pool of Postgres connections and multiplexes requests onto them.

Modes

- **session** — client gets a connection until disconnect. Lowest pooling benefit.
- **transaction** — client gets a connection per transaction. **Most useful**.
- **statement** — client gets a connection per statement. Highest pooling but breaks anything that uses transactions (most apps).

For web apps, use **transaction mode**.

Docker setup

In your VPS's `infra/docker-compose.yml`:

```
services:
  postgres:
    # ... existing
  pgbouncer:
    image: edoburu/pgbouncer:latest
    environment:
      DATABASES_HOST: postgres
      DATABASES_PORT: 5432
      DATABASES_USER: postgres
      DATABASES_PASSWORD: ${POSTGRES_ROOT_PASSWORD}
      POOL_MODE: transaction
      MAX_CLIENT_CONN: 1000
      DEFAULT_POOL_SIZE: 25
    networks:
      - shared
    depends_on:
      postgres:
        condition: service_healthy
```

Apps connect to `pgbouncer:5432` instead of `postgres:5432`. Same wire protocol.

Gotchas

- Transaction mode breaks `SET` (session variables), `LISTEN/NOTIFY`, prepared statements (kinda — needs `server_reset_query`).
- Per-database connection pools — pgbouncer maintains separate pools per `(db, user)` combination.

Skip pgbouncer for: low-concurrency apps (Discord bots, scheduled jobs). Use it for: anything web-facing with many concurrent requests.

19. Replication concepts

You probably don't need replication on a personal VPS. But the vocabulary is good to know.

Physical replication (streaming)

Replica streams WAL from primary and replays it. Replica is byte-identical to primary.

- **Synchronous** — primary waits for replica to confirm before committing. Strongest durability, latency cost.
- **Asynchronous** — primary commits and replica catches up later. Default. Some data loss possible on primary failure.

Use for: read replicas, hot standby, disaster recovery (separate server, separate region).

Logical replication

Replica receives row-level changes (INSERT/UPDATE/DELETE) rather than raw WAL. Different Postgres versions can replicate; you can replicate a subset of tables.

Use for: cross-version migrations, partial replication, multi-master (complex), CDC pipelines.

Setup

Setting up replication is a separate doc. For personal use: **backups (covered in backups-and-restore) are your replication.**

20. Anti-patterns to know and avoid

Storing time as TEXT

```
-- DON'T
created_at TEXT NOT NULL      -- "2026-05-27 14:30:00"

-- DO
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

You lose: time math, indexing, timezone awareness.

Money as FLOAT

```
-- DON'T
price FLOAT
-- 0.1 + 0.2 != 0.3 disasters

-- DO
price_cents BIGINT NOT NULL
-- or
price NUMERIC(12, 2) NOT NULL
```

Soft delete without partial index

```
-- DON'T just have:
CREATE INDEX idx_users_email ON users(email);

-- DO (when 99% of queries are "WHERE deleted_at IS NULL")
CREATE UNIQUE INDEX uq_users_email_active ON users(email) WHERE deleted_at IS NULL;
```

One giant `data` JSONB column instead of any schema

```
-- DON'T
CREATE TABLE everything (
  id BIGSERIAL PRIMARY KEY,
  data JSONB NOT NULL      -- "It's flexible!"
);

-- DO: model what you know; use JSONB only for genuinely variable fields
```

You lose: foreign keys, NOT NULL guarantees, type checking, query performance, ability to grep schema for what your app cares about.

N+1 query patterns at the app layer

```
# DON'T
users = db.execute("SELECT * FROM users").all()
for u in users:
    posts = db.execute("SELECT * FROM posts WHERE user_id = ?", u.id).all()
    # N+1 queries

# DO
rows = db.execute("""
    SELECT u.id, u.name, p.title
    FROM users u
    LEFT JOIN posts p ON p.user_id = u.id
""").all()
```

Most ORMs have a "preload" / "eager load" feature. Use it.

Missing FK indexes

Easy to forget. Always:

```
CREATE TABLE posts (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id),
    -- ...
);
CREATE INDEX idx_posts_user_id ON posts(user_id);    -- don't forget!
```

Without the index, deleting a user does a sequential scan of `posts` for the FK check.

Using `SERIAL` instead of `IDENTITY`

```
-- Legacy (still works, but)
id SERIAL PRIMARY KEY

-- Modern (PG 10+)
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

`SERIAL` has subtle issues with role grants and sequence ownership. `IDENTITY` is cleaner.

Connecting as superuser from your app

```
-- DON'T have your app's DATABASE_URL point to postgres user
-- DO create a per-app user with only the privileges it needs
CREATE USER appuser WITH PASSWORD '...';
CREATE DATABASE appdb OWNER appuser;
```

Limit damage from app compromises. See [vps-from-zero §15](#).

Forgetting `created_at` and `updated_at`

Every table. Always. You will thank yourself.

```
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

Plus a trigger for `updated_at` (or use ORM features that do it).

Allowing NULL when "" or 0 would do

NULL has three-valued logic. It changes how indexes behave, how comparisons work, how aggregates count. Reach for it only when "I don't know" is genuinely different from "no value."

21. Cheat sheet

Connection

```
psql -h host -p 5432 -U user -d db
psql 'postgres://user:pass@host/db'
docker compose exec postgres psql -U postgres
```

Essential meta-commands

```
\l          \dt          \d table      \du          \dn          \dx
\timing     \x auto      \pset null 'Ø' \e          \i file      \q
```

Schema inspection

```
SELECT pg_size_pretty(pg_database_size('mydb'));
SELECT pg_size_pretty(pg_total_relation_size('users'));
\d+ users
\di+ users          -- indexes on users
```

Index types

Type	For
B-tree	Equality, range, ORDER BY (default)
Hash	Equality only
GIN	Arrays, JSONB, tsvector
GiST	Ranges, geometry, exclusion
BRIN	Huge, naturally-ordered tables
Partial	<code>WHERE</code> clause filters most queries
Expression	Indexed function/expression
Covering	INCLUDE columns in leaf

Query helpers

```
-- UPSERT
INSERT INTO t (k, v) VALUES ('a', 1)
ON CONFLICT (k) DO UPDATE SET v = EXCLUDED.v;

-- Bulk values
INSERT INTO t (k) VALUES ('a'), ('b'), ('c');

-- RETURNING
INSERT INTO t (k) VALUES ('a') RETURNING id;

-- DISTINCT ON
SELECT DISTINCT ON (user_id) * FROM orders ORDER BY user_id, created_at DESC;

-- LATERAL top-N
SELECT u.*, p.*
FROM users u
CROSS JOIN LATERAL (
    SELECT * FROM posts WHERE user_id = u.id
    ORDER BY created_at DESC LIMIT 3
) p;

-- Recursive CTE
WITH RECURSIVE x AS (
    SELECT base_row UNION ALL
    SELECT next_row FROM x JOIN t ON ...
) SELECT * FROM x;

-- FILTER
SELECT count(*) FILTER (WHERE status = 'paid') FROM orders;
```

JSONB

```
->                -- field as jsonb
->>                -- field as text
#>                -- path as jsonb
#>>               -- path as text
@>                -- contains
<@                -- contained by
?                 -- key exists
?|                -- any key
?&                -- all keys
@?                -- jsonpath exists
@@                -- jsonpath predicate

-- Indexes
CREATE INDEX ix ON t USING GIN (col);
CREATE INDEX ix ON t USING GIN (col jsonb_path_ops);
CREATE INDEX ix ON t ((col->>'field'));
```

EXPLAIN

```
EXPLAIN ANALYZE SELECT ...;
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...;
```

Watch for: Seq Scan on large table, big estimated-vs-actual row count gap, Sort with high cost.

Transactions

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
-- work
COMMIT;
-- or
ROLLBACK;

-- Job queue pattern:
SELECT * FROM jobs WHERE status='pending'
ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 1;
```

Useful pg_catalog queries

```
-- All tables by size
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS sz
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;

-- Long-running queries
SELECT pid, age(clock_timestamp(), query_start) AS age, query
FROM pg_stat_activity WHERE state != 'idle'
AND query_start < now() - interval '5 seconds';

-- Slow query stats (needs pg_stat_statements)
SELECT round(total_exec_time::numeric, 2) AS total_ms,
       calls, round(mean_exec_time::numeric, 2) AS mean_ms,
       substring(query, 1, 80)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;

-- Dead row pct per table
SELECT relname, n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / NULLIF(n_live_tup+n_dead_tup, 0), 1) AS dead_pct
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;

-- Currently held locks
SELECT pid, locktype, mode, relation::regclass AS table
FROM pg_locks WHERE granted ORDER BY pid;

-- Kill a query
SELECT pg_terminate_backend(PID);
```

Best-default `~/.psqlrc`

```
\set QUIET 1
\x auto
\timing on
\pset null 'ø'
\pset linestyle unicode
\pset border 2
\set HISTFILE ~/.psql_history- :DBNAME
\set HISTCONTROL ignoredups
\set HISTSIZE 5000
\set COMP_KEYWORD_CASE upper
\unset QUIET
```

Table template

```
CREATE TABLE example (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

  -- business columns here, NOT NULL by default
  name TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active', 'inactive', 'archived')),

  -- foreign keys with explicit ON DELETE
  parent_id BIGINT REFERENCES parents(id) ON DELETE CASCADE,

  -- timestamps, always
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_example_parent_id ON example(parent_id);

-- updated_at trigger
CREATE TRIGGER trg_example_updated_at
  BEFORE UPDATE ON example
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```